

P0462R0: Marking `memory_order_consume` Dependency Chains

Doc. No.: WG21/P0462R0

Date: 2016-10-13

Reply to: Paul E. McKenney, Torvald Riegel, Jeff Preshing,
Hans Boehm, Clark Nelson, Olivier Giroux, Lawrence Cowl,
JF Bastien, and Micheal Wong

Email: paulmck@linux.vnet.ibm.com, triegel@redhat.com, jeff@preshing.com
boehm@acm.org, clark.nelson@intel.com, OGiroux@nvidia.com,
Lawrence@Cowl.org, jfbastien@apple.com, and fraggamuffin@gmail.com

Other contributors: Alec Teal, Alisdair Meredith, David Howells, David Lang,
George Spelvin, Jeff Law, Joseph S. Myers, Linus Torvalds, Mark Batty, Michael Matz,
Peter Sewell, Peter Zijlstra, Ramana Radhakrishnan, Richard Biener, Will Deacon,
Faisal Vali, Behan Webster, Tony Tye, Thomas Koeppel, Jens Maurer, ...

October 13, 2016

This document is based in part on WG21/D0098R1, extracting the alternatives for marking dependency chains (each headed by a `memory_order_consume` [1] load). It also adds a few additional alternatives based on discussions at the March 2016 meeting in Jacksonville, Florida. This document does not define the behavior of dependency chains, but instead only the syntax used to call the compiler's attention to them. Please see WG21/P0190R2 for detailed information on dependency-chain behavior.

1 Introduction

Spirited discussions of `memory_order_consume` at the Jacksonville meeting resulted in a few of items of agreement:

1. Dependency chains should be restricted to pointers. Please note that this excludes not only the

troublesome objects of integral type, but also accesses to static members of classes.

2. Unmarked code can be handled by having the implementation behave as if markings had been supplied in all locations that could reasonably be marked. This allows natural handling of dependencies in unmarked code. This behavior should be controlled by a compiler flag. Such a flag is of course outside of the standard.
3. Software artifacts that are built standalone (such as the Linux kernel and numerous embedded projects) can reasonably use unmarked dependency chains. In contrast, software artifacts that are expected to dynamically link against standard libraries seem likely to need to mark their dependency chains.
4. Discussions involving marking of library APIs have been set aside for the moment, and so this

document does not address this point.

These points result in three known valid ways of handling `memory_order_consume`:

1. Ignore the markings and promote `memory_order_consume` to `memory_order_acquire`, as is current practice.
2. Ignore the markings, demote `memory_order_consume` to `memory_order_relaxed`, and suppress troublesome optimizations of pointers. However, there was some difficulty in arriving at a precise definition of “troublesome”.
3. Demote `memory_order_consume` to `memory_order_relaxed` and suppress troublesome optimizations of marked pointers. The fact that such optimizations need not be suppressed for unmarked pointers means that a much more conservative definition of “troublesome” is feasible, thus reducing the need for precision. Note that pointer comparisons will still break dependency chains in some cases, unless the comparisons were carried out using proposed dependency-preserving pointer-comparison intrinsics. Note further that the template-based method described in Section 3.4.4 uses operator overloading so that the usual relational operators invoke these intrinsics.

However, a number of ways of marking dependency chains have been proposed and there was nothing resembling any sort of agreement on which should be used. This paper therefore catalogs approaches to marking dependency chains, and evaluates each against a set of representative use cases.

2 Representative Use Cases

This section uses the common definitions shown in Figure 1 to discuss the use cases in the following list:

1. Simple case.
2. Function in via parameter.
3. Function out via return value.

```

1 struct rcutest {
2     int a;
3     int b;
4     int c;
5     spinlock_t lock;
6 };
7
8 struct rcutest1 {
9     int a;
10    struct rcutest rt;
11 };
12
13 std::atomic<rcutest *> gp;
14 std::atomic<rcutest1 *> g1p;
15 std::atomic<int *> gip;
16 struct rcutest *gslp; /* Global scope, local usage. */
17 std::atomic<rcutest *> gsgp;
18
19 template<typename T>
20 T *rcu_consume(std::atomic<T*> *p)
21 {
22     volatile std::atomic<T*> *q = p;
23     // Change to memory_order_consume once it is fixed
24     depending_ptr<T> temp(q->load(std::memory_order_relaxed));
25     return temp;
26 }
27
28 template<typename T>
29 T *rcu_consume(T *p)
30 {
31     // Alternatively, could cast p to volatile atomic...
32     T *temp(*(T *volatile *)&p);
33     return temp;
34 }
35
36 template<typename T>
37 T *rcu_store_release(std::atomic<T*> *p, T *v)
38 {
39     p->store(v, std::memory_order_release);
40     return v;
41 }
42
43 template<typename T>
44 T *rcu_store_release(T **p, T *v)
45 {
46     // Alternatively, could cast p to volatile atomic...
47     atomic_thread_fence(std::memory_order_release);
48     *((volatile T **)p) = v;
49     return v;
50 }
51
52 // Linux-kernel compatibility macros, not for atomics
53 #define rcu_dereference(p) rcu_consume(p)
54 #define rcu_assign_pointer(p, v) rcu_store_release(&p, v)

```

Figure 1: Common Definitions

```

1 void *thread0(void *unused)
2 {
3     rcutest *p;
4
5     p = new rcutest();
6     assert(p);
7     p->a = 42;
8     assert(p->a != 43);
9     rcu_store_release(&gp, p);
10    return nullptr;
11 }
12
13 void *thread1(void *unused)
14 {
15     rcutest p;
16
17     p = rcu_consume(&gp);
18     if (p)
19         p->a = 43;
20     return nullptr;
21 }

```

Figure 2: Simple Case

4. Function both in and out, but different chains.
5. Dependency chain fanning out.
6. Dependency chain fanning in.
7. Dependency chain fanning both in and out.
8. Conditional compilation of endpoint accesses.
9. Examples involving handoff to locking.

Each of the above use cases is covered in one of the following sections, followed by a discussion of evaluation criteria.

2.1 Simple Case

The simple case is shown in Figure 2. Here, the dependency chain extends from line 16 through line 18, where it terminates. Given the simplicity and compactness of this example, any reasonable proposal should handle this example simply and naturally.

2.2 In via Function Parameter

Figure 3 shows an example dependency chain that begins at line 22, enters function `thread1_help()` at line 23, and then extending from line 12 to line 15 in the called function. This is a common encapsulation technique.

```

1 void thread0(void)
2 {
3     struct rcutest *p;
4
5     p = new rcutest;
6     assert(p);
7     p->a = 42;
8     rcu_assign_pointer(gp, p);
9 }
10
11 void
12 thread1_help(struct rcutest *q)
13 {
14     if (q)
15         assert(q->a == 42);
16 }
17
18 void thread1(void)
19 {
20     struct rcutest *p;
21
22     p = rcu_dereference(gp);
23     thread1_help(p);
24 }

```

Figure 3: In via Function Parameter

2.3 Out via Function Return

Figure 4 shows a dependency chain exiting a function. It starts at line 21, is returned to line 20, and terminates on line 22. This is also a common encapsulation technique.

2.4 In and Out, But Different Chains

Figure 5 shows an example where a dependency chain enters a function (`thread1_help()` on lines 16-21) and a dependency chain leaves that same function, but where they are different chains.

2.5 Chain Fanning Out

Figure 6 shows a dependency chain fanning out, courtesy of the `thread1()` function's calls to `thread1_help1()` and `thread1_help2()` on lines 30 and 31. This is a common pattern in the Linux kernel, as it supports abstraction of data structures, for example, allowing common RCU-protected data structures to be aggregated into a larger RCU-protected data structure. In this scenario, `thread1_help1()` might implement one type of RCU-protected structure and `thread1_help2()` might implement another.

```

1 void thread0(void)
2 {
3     struct rcutest *p;
4
5     p = new rcutest;
6     assert(p);
7     p->a = 42;
8     rcu_assign_pointer(gp, p);
9 }
10
11 struct rcutest *thread1_help(void)
12 {
13     return rcu_dereference(gp);
14 }
15
16 void thread1(void)
17 {
18     struct rcutest *p;
19
20     p = thread1_help();
21     if (p)
22         assert(p->a == 42);
23 }

```

Figure 4: Out via Function Return

```

1 void thread0(void)
2 {
3     struct rcutest *p;
4
5     p = new rcutest;
6     assert(p);
7     p->a = 42;
8     rcu_assign_pointer(gp, p);
9
10    p = new rcutest;
11    assert(p);
12    p->a = 43;
13    rcu_assign_pointer(gsgp, p);
14 }
15
16 struct rcutest *thread1_help(struct rcutest *p)
17 {
18     if (p)
19         assert(p->a == 42);
20     return rcu_dereference(gsgp);
21 }
22
23 void thread1(void)
24 {
25     struct rcutest *p;
26
27     p = rcu_dereference(gp);
28     p = thread1_help(p);
29     if (p)
30         assert(p->a == 43);
31 }

```

Figure 5: In and Out, But Different Chains

```

1 void thread0(void)
2 {
3     struct rcutest *p;
4
5     p = new rcutest;
6     assert(p);
7     p->a = 42;
8     rcu_assign_pointer(gp, p);
9 }
10
11 void
12 thread1_help1(struct rcutest *q)
13 {
14     if (q)
15         assert(q->a == 42);
16 }
17
18 void
19 thread1_help2(struct rcutest *q)
20 {
21     if (q)
22         assert(q->a != 43);
23 }
24
25 void thread1(void)
26 {
27     struct rcutest *p;
28
29     p = rcu_dereference(gp);
30     thread1_help1(p);
31     thread1_help2(p);
32 }

```

Figure 6: Chain Fanning Out

```

1 void thread0(void)
2 {
3     struct rcutest *p;
4     struct rcutest1 *p1;
5
6     p = new rcutest;
7     assert(p);
8     p->a = 42;
9     rcu_assign_pointer(gp, p);
10
11    p1 = new rcutest;
12    assert(p1);
13    p1->a = 41;
14    p1->rt.a = 42;
15    rcu_assign_pointer(g1p, p1);
16 }
17
18 void
19 thread1_help(struct rcutest *q)
20 {
21     if (q)
22         assert(q->a == 42);
23 }
24
25 void thread1(void)
26 {
27     struct rcutest *p;
28
29     p = rcu_dereference(gp);
30     thread1_help(p);
31 }
32
33 void thread2(void)
34 {
35     struct rcutest1 *p1;
36
37     p1 = rcu_dereference(g1p);
38     thread1_help(&p1->rt);
39 }

```

Figure 7: Chain Fanning In

2.6 Chain Fanning In

Figure 7 demonstrates different dependency chains fanning into the same function, in this case `thread1_help()`, from lines 29 and 37. This fanning-in is also used to support abstraction, for example, allowing a given implementation of an RCU-protected data structure to be aggregated into several different data structures.

2.7 Chain Fanning In and Out

Figure 8 shows dependency chains fanning both in and out, starting at lines 45 and 53, fanning into `thread1_help()`, and fanning out again at the call to `thread1a_help()` on line 36 and to `thread1b_help()`

```

1 void thread0(void)
2 {
3     struct rcutest *p;
4     struct rcutest1 *p1;
5
6     p = new rcutest;
7     assert(p);
8     p->a = 42;
9     p->b = 43;
10    rcu_assign_pointer(gp, p);
11
12    p1 = new rcutest;
13    assert(p1);
14    p1->a = 41;
15    p1->rt.a = 42;
16    p1->rt.b = 43;
17    rcu_assign_pointer(g1p, p1);
18 }
19
20 void
21 thread1a_help(struct rcutest *q)
22 {
23     assert(q->a == 42);
24 }
25
26 void
27 thread1b_help(struct rcutest *q)
28 {
29     assert(q->b == 43);
30 }
31
32 void
33 thread1_help(struct rcutest *q)
34 {
35     if (q) {
36         thread1a_help(q);
37         thread1b_help(q);
38     }
39 }
40
41 void thread1(void)
42 {
43     struct rcutest *p;
44
45     p = rcu_dereference(gp);
46     thread1_help(p);
47 }
48
49 void thread2(void)
50 {
51     struct rcutest1 *p1;
52
53     p1 = rcu_dereference(g1p);
54     thread1_help(&p1->rt);
55 }

```

Figure 8: Chain Fanning In and Out

on line 37. This combination permits composition of the types of abstraction described in Sections 2.5 and 2.6.

2.8 Conditional Compilation of Chain Endpoints

Although the C preprocessor does not necessarily have the best reputation among the various aspects of either C or C++, it is true that it is always there when you need it. Figure 9 applies conditional compilation to Figure 8, so that portions of the dependency chain can come and go, depending on the value of the C-preprocessor macro `F00`.

2.9 Handoff to Locking

Figure 10 shows how RCU protection can hand off to other synchronization primitives, in this case, locking. The dependency chain starts at line 16 and continues through line 18 and 19. However, once line 19 has completed, the code is under the protection of `p->lock`, so line 20 explicitly ends the dependency chain. The lock then protects the increment on line 21.

It is also possible to hand off protection from RCU to reference counting, explicit memory barriers, transactional memory, and so on.

Note that the `std::kill_dependency()` on line 20 will typically have no effect on code generation.

2.10 Evaluation Criteria

1. Ease of compilation.
2. Ease of modification of programs.
3. Precise specification of dependency chains.
4. Support for cross-function dependency chains.
5. Support for cross-compilation-unit dependency chains.
6. Compatibility with C.
7. Formal Verification Compatibility.

```

1 void thread0(void)
2 {
3     struct rcutest *p;
4     struct rcutest1 *p1;
5
6     p = new rcutest;
7     assert(p);
8     p->a = 42;
9     p->b = 43;
10    rcu_assign_pointer(gp, p);
11
12    p1 = new rcutest;
13    assert(p1);
14    p1->a = 41;
15    p1->rt.a = 42;
16    p1->rt.b = 43;
17    rcu_assign_pointer(g1p, p1);
18 }
19
20 #ifdef F00
21 void
22 thread1a_help(struct rcutest *q)
23 {
24     assert(q->a == 42);
25 }
26 #endif
27
28 void
29 thread1b_help(struct rcutest *q)
30 {
31     assert(q->b == 43);
32 }
33
34 void
35 thread1_help(struct rcutest *q)
36 {
37     if (q) {
38         #ifdef F00
39             thread1a_help(q);
40         #endif
41         thread1b_help(q);
42     }
43 }
44
45 void thread1(void)
46 {
47     struct rcutest *p;
48
49     p = rcu_dereference(gp);
50     thread1_help(p);
51 }
52
53 void thread2(void)
54 {
55     struct rcutest1 *p1;
56
57     p1 = rcu_dereference(g1p);
58     thread1_help(&p1->rt);
59 }

```

Figure 9: Conditional Compilation of Chain Endpoints

```

1 void thread0(void)
2 {
3     struct rcutest *p;
4
5     p = new rcutest;
6     assert(p);
7     p->a = 42;
8     assert(p->a != 43);
9     rcu_assign_pointer(gp, p);
10 }
11
12 void thread1(void)
13 {
14     struct rcutest *p;
15
16     p = rcu_dereference(gp);
17     if (p) {
18         assert(p->a == 42);
19         spin_lock(&p->lock);
20         p = std::kill_dependency(p);
21         p->a++;
22         spin_unlock(&p->lock);
23     }
24 }

```

Figure 10: Handoff to Locking

3 Marking Proposals

The following sections present alternative marking proposals. Whatever proposal is chosen, implementors are encouraged to provide a means (for example, a command-line argument) to cause the implementation to act as if markings were placed everywhere they could reasonably be placed. This approach permits unmarked programs containing dependency chains to be handled in a reasonably natural manner.

Note that many of these proposals consist only of short descriptions. Only proposals having proponents willing to fill them out should be considered for standardization.

3.1 Mark Translation Unit

Within the language, translation-unit marking could be accomplished by a pragma or by a language feature that changed the way pointers are implemented. A compiler command-line argument could also be used, but this is of course outside the standard. It would be desirable for marked translation units to be able to be linked with unmarked translation units.

This approach could be useful in cases where only a few of the translation units contain dependency

chains. However, software-engineering considerations would likely cause many such projects to mark all the translation units, which would of course result in the same dependency-chain-tracing complexity as would unmarked dependency chains. Any full proposal for this approach should therefore describe how this issue will be handled.

3.2 Mark Range of Code

Ranges of code could be marked by pragmas, through use of C preprocessor symbols, or via other ad-hoc means. However, again, software-engineering considerations would likely cause many such projects to mark all the translation units, which would of course result in the same dependency-chain-tracing complexity as would unmarked dependency chains. Any full proposal for this approach should therefore describe how this issue will be handled.

3.3 Mark Functions

Functions containing dependency chains could be marked with an attribute (for example, something like `[[function_carries_dependencies]]`) or a keyword (for example, something like `-FunctionCarriesDependencies`).

Proper use of this approach eliminates issues with dependencies passing through dependency-unaware code: Simply mark the relevant functions. However, although there are many software-engineering reasons for preferring small functions, the fact remains that large functions are not uncommon in production code. Large marked functions of course result in similar dependency-chain-tracing complexity as would unmarked code, so any full proposal for this approach should describe how this tracing will be handled.

3.4 Mark Objects

This class of proposals marks the objects that are to carry dependencies. These objects must be of pointer type. Note that implementations requiring point-to-point associations between each `memory_order_consume` load and its corresponding dependent memory references can generate these associations based

on the operations carried out on a given marked object.

3.4.1 Attribute

This approach, suggested by Clark Nelson, generalizes the `[[carries_dependency]]` attribute specified in the C++11 standard so that it applies to objects, including variables, formal parameters, return values, and class members. This paper further modifies this proposed attribute so as to also restrict it to pointer-like objects.

There have been some objections to attributes on the grounds that attributes are not supposed to change program semantics, but no consensus as to whether or not this objection is substantive.

The changes to the examples from Section 2 are similar to those shown in Section 3.4.3.

3.4.2 Type Qualifier

This approach, put forward by Torvald Riegel in response to Linus Torvalds's spirited criticisms of the current C11 and C++11 wording, introduces a new `value_dep_preserving` type qualifier. Objects marked with this type qualifier carry dependencies.

Again, the changes to the examples from Section 2 are similar to those shown in Section 3.4.3.

3.4.3 Object Modifier

This approach uses a keyword that does not participate in type checking, for example, a `_Carries_dependency` keyword. This might be treated in a manner similar to a storage class. It need not necessarily interact with the type system.

Figures 11–19 show how object modifiers can be applied to each of the examples introduced in Section 2. These changes are straightforward markings of local variables, function parameters, and return-value types. Object modifiers therefore easily support the use cases in the Linux kernel.¹

¹ Give or take a strong distaste for any sort of marking scheme on the part of numerous Linux-kernel community members.

```

1 void thread0()
2 {
3     rcutest *p = new rcutest();
4     p->a = 42;
5     assert(p->a != 43);
6     rcu_assign_pointer(gp, p);
7 }
8
9 void thread1()
10 {
11     rcutest _Carries_dependency *p = rcu_dereference(gp);
12     if (p)
13         p->a = 43;
14 }

```

Figure 11: Object Modifier: Simple Case

```

1 void thread0()
2 {
3     rcutest *p = new rcutest();
4     p->a = 42;
5     rcu_assign_pointer(gp, p);
6 }
7
8 void
9 thread1_help(rcutest _Carries_dependency *q)
10 {
11     if (q)
12         assert(q->a == 42);
13 }
14
15 void thread1()
16 {
17     rcutest _Carries_dependency *p = rcu_dereference(gp);
18     thread1_help(p);
19 }

```

Figure 12: Object Modifier: In via Function Parameter

```

1 void thread0()
2 {
3     rcutest *p = new rcutest();
4     p->a = 42;
5     rcu_assign_pointer(gp, p);
6 }
7
8 rcutest _Carries_dependency *thread1_help()
9 {
10     return rcu_dereference(gp);
11 }
12
13 void thread1()
14 {
15     rcutest _Carries_dependency *p = thread1_help();
16     if (p)
17         assert(p->a == 42);
18 }

```

Figure 13: Object Modifier: Out via Function Return


```

1 void thread0()
2 {
3   rcutest *p = new rcutest();
4   p->a = 42;
5   rcu_assign_pointer(gp, p);
6
7   p = new rcutest();
8   p->a = 43;
9   rcu_assign_pointer(gsgp, p);
10 }
11
12 rcutest _Carries_dependency *
13 thread1_help(rcutest _Carries_dependency *p)
14 {
15   if (p)
16     assert(p->a == 42);
17   return rcu_dereference(gsgp);
18 }
19
20 void thread1(void)
21 {
22   rcutest _Carries_dependency *p = rcu_dereference(gp);
23   p = thread1_help(p);
24   if (p)
25     assert(p->a == 43);
26 }

```

Figure 14: Object Modifier: In and Out, But Different Chains

```

1 void thread0()
2 {
3   rcutest *p = new rcutest();
4   p->a = 42;
5   rcu_assign_pointer(gp, p);
6   rcutest1 *p1 = new rcutest1();
7   p1->a = 41;
8   p1->rt.a = 42;
9   rcu_assign_pointer(g1p, p1);
10 }
11
12 void
13 thread1_help(rcutest _Carries_dependency *q)
14 {
15   if (q)
16     assert(q->a == 42);
17 }
18
19 void thread1()
20 {
21   rcutest _Carries_dependency *p = rcu_dereference(gp);
22   thread1_help(p);
23 }
24
25 void thread2()
26 {
27   rcutest1 _Carries_dependency *p1 = rcu_dereference(g1p);
28   thread1_help(&p1->rt);
29 }

```

Figure 16: Object Modifier: Chain Fanning In

```

1 void thread0()
2 {
3   rcutest *p = new rcutest();
4   p->a = 42;
5   rcu_assign_pointer(gp, p);
6 }
7
8 void
9 thread1_help1(rcutest _Carries_dependency *q)
10 {
11   if (q)
12     assert(q->a == 42);
13 }
14
15 void
16 thread1_help2(rcutest _Carries_dependency *q)
17 {
18   if (q)
19     assert(q->a != 43);
20 }
21
22 void thread1()
23 {
24   rcutest _Carries_dependency *p = rcu_dereference(gp);
25   thread1_help1(p);
26   thread1_help2(p);
27 }

```

Figure 15: Object Modifier: Chain Fanning Out

3.4.4 Template

This approach, suggested off-list by JF Bastien, creates a `depending_ptr`² template to which a pointer-like type is passed. This approach allows implementers considerable freedom, as they can hook into the `->` and `*` if need be, and also use the C++ `delete` keyword to prohibit problematic operations. Implementations that might nevertheless carry out aggressive optimizations that might break dependencies even for the non-problematic operations might need to implement this template class in a manner similar to the atomics template classes.

This approach would need to be augmented with a non-template solution for C, for example, the object-modifier approach from Section 3.4.3. Implementations that support both C and C++ would presumably relate Section 3.4.3's keyword to the templates in this section in a manner similar to that used for atomics.

Figure 20 shows the resulting template declaration,

² Arbitrarily chosen name with no Google hits.

```

1 void thread0()
2 {
3   rcutest *p = new rcutest();
4   p->a = 42;
5   p->b = 43;
6   rcu_assign_pointer(gp, p);
7   rcutest1 *p1 = new rcutest1();
8   p1->a = 41;
9   p1->rt.a = 42;
10  p1->rt.b = 43;
11  rcu_assign_pointer(g1p, p1);
12 }
13
14 void
15 thread1a_help(rcutest _Carries_dependency *q)
16 {
17   assert(q->a == 42);
18 }
19
20 void
21 thread1b_help(rcutest _Carries_dependency *q)
22 {
23   assert(q->b == 43);
24 }
25
26 void
27 thread1_help(rcutest _Carries_dependency *q)
28 {
29   if (q) {
30     thread1a_help(q);
31     thread1b_help(q);
32   }
33 }
34
35 void thread1()
36 {
37   rcutest _Carries_dependency *p = rcu_dereference(gp);
38   thread1_help(p);
39 }
40
41 void thread2()
42 {
43   rcutest1 _Carries_dependency *p1 = rcu_dereference(g1p);
44   thread1_help(&p1->rt);
45 }

```

Figure 17: Object Modifier: Chain Fanning In and Out

```

1 void thread0()
2 {
3   struct rcutest *p = new rcutest();
4   p->a = 42;
5   p->b = 43;
6   rcu_assign_pointer(gp, p);
7   struct rcutest1 *p1 = new rcutest1();
8   p1->a = 41;
9   p1->rt.a = 42;
10  p1->rt.b = 43;
11  rcu_assign_pointer(g1p, p1);
12 }
13
14 #ifdef F00
15 void
16 thread1a_help(rcutest _Carries_dependency *q)
17 {
18   assert(q->a == 42);
19 }
20 #endif
21
22 void
23 thread1b_help(rcutest _Carries_dependency *q)
24 {
25   assert(q->b == 43);
26 }
27
28 void
29 thread1_help(rcutest _Carries_dependency *q)
30 {
31   if (q) {
32     #ifdef F00
33       thread1a_help(q);
34     #endif
35     thread1b_help(q);
36   }
37 }
38
39 void thread1()
40 {
41   rcutest _Carries_dependency *p = rcu_dereference(gp);
42   thread1_help(p);
43 }
44
45 void thread2()
46 {
47   rcutest1 _Carries_dependency *p1 = rcu_dereference(g1p);
48   thread1_help(&p1->rt);
49 }

```

Figure 18: Object Modifier: Conditional Compilation of Chain Endpoints

```

1 void thread0()
2 {
3     rcutest *p = new rcutest();
4     p->a = 42;
5     assert(p->a != 43);
6     rcu_assign_pointer(gp, p);
7 }
8
9 void thread1()
10 {
11     rcutest _Carries_dependency *p = rcu_dereference(gp);
12     if (p) {
13         assert(p->a == 42);
14         spin_lock(&p->lock);
15         p = std::kill_dependency(p);
16         p->a++;
17         spin_unlock(&p->lock);
18     }
19 }
20 }

```

Figure 19: Object Modifier: Handoff to Locking

each member function of which has a straightforward definition. Note especially that the relational operators are defined in terms of the `pointer_cmp_eq_dep()`, `pointer_cmp_ne_dep()`, `pointer_cmp_gt_dep()`, `pointer_cmp_ge_dep()`, `pointer_cmp_lt_dep()`, and `pointer_cmp_le_dep()` functions shown in Figure 21, so that as long as the first argument to a relational operator is of type `class depending_ptr<T>`, pointers may be safely compared without risk of breaking dependency chains.³ In addition, the operators that cannot be safely applied to dependency-bearing pointers are omitted.⁴ Finally, Figure 22 shows how the Linux-kernel-style `rcu_dereference()` and `rcu_assign_pointer()` macros could be implemented given this templated approach.

Figures 23–31 show how templates can be applied to each of the examples introduced in Section 2. As with the object-modifier approach in Section 3.4.3, these changes are straightforward markings of local variables, function parameters, and return-value types.

Full source code for a prototype implementation (and for this paper) may be downloaded from

³ That said, in the prototype implementation, these are not intrinsics, but rather separately compiled functions. In the absence of link-time optimizations, separate compilation preserves dependency chains in most implementations.

⁴ The number of pointer-tagging algorithms should motivate allowing bitwise operations on dependency-bearing pointers, but this should be handled separately.

```

1 template<typename T>
2 class depending_ptr {
3 public:
4     typedef T* pointer;
5     typedef T element_type;
6
7     // Constructors
8     constexpr depending_ptr() noexcept;
9     explicit depending_ptr(T* v) noexcept;
10    depending_ptr(nullptr_t) noexcept;
11    depending_ptr(const depending_ptr &d) noexcept;
12    depending_ptr(const depending_ptr &&d) noexcept;
13
14    // Assignment
15    depending_ptr& operator=(pointer p) noexcept;
16    depending_ptr& operator=(const depending_ptr &d) noexcept;
17    depending_ptr& operator=(const depending_ptr &&d) noexcept;
18    depending_ptr& operator=(nullptr_t) noexcept;
19
20    // Modifiers
21    void swap(depending_ptr& d) noexcept;
22
23    // Unary operators
24    // No operator!
25    // No prefix bitwise complement operator
26    element_type operator*() noexcept;
27    pointer operator->() noexcept;
28    depending_ptr<element_type> operator++();
29    depending_ptr<element_type> operator++(int);
30    depending_ptr<element_type> operator--();
31    depending_ptr<element_type> operator--(int);
32    pointer get() const noexcept;
33    explicit operator bool();
34    element_type operator[](size_t);
35
36    // Binary relational operators
37    bool operator==(depending_ptr v) noexcept;
38    bool operator!=(depending_ptr v) noexcept;
39    bool operator>(depending_ptr v) noexcept;
40    bool operator>=(depending_ptr v) noexcept;
41    bool operator<(depending_ptr v) noexcept;
42    bool operator<=(depending_ptr v) noexcept;
43    bool operator==(pointer v) noexcept;
44    bool operator!=(pointer v) noexcept;
45    bool operator>(pointer v) noexcept;
46    bool operator>=(pointer v) noexcept;
47    bool operator<(pointer v) noexcept;
48    bool operator<=(pointer v) noexcept;
49
50    // Other binary operators
51    depending_ptr<T> operator+(size_t idx);
52    depending_ptr<T> operator+=(size_t idx);
53    depending_ptr<T> operator-(size_t idx);
54    depending_ptr<T> operator-=(size_t idx);
55
56 private:
57     pointer dp_rep;
58 };

```

Figure 20: Template: Declaration

```

1 bool pointer_cmp_eq_dep(void *p, void *q) noexcept;
2 bool pointer_cmp_ne_dep(void *p, void *q) noexcept;
3 bool pointer_cmp_gt_dep(void *p, void *q) noexcept;
4 bool pointer_cmp_ge_dep(void *p, void *q) noexcept;
5 bool pointer_cmp_lt_dep(void *p, void *q) noexcept;
6 bool pointer_cmp_le_dep(void *p, void *q) noexcept;

```

Figure 21: Dependency-Preserving Comparisons

```

1 template<typename T>
2 depending_ptr<T> rcu_consume(std::atomic<T*> *p)
3 {
4     volatile std::atomic<typename
5         depending_ptr<T>::pointer> *q = p;
6     // Change to memory_order_consume once it is fixed
7     depending_ptr<T> temp(q->load(std::memory_order_relaxed));
8
9     return temp;
10 }
11
12 template<typename T>
13 depending_ptr<T> rcu_consume(T *p)
14 {
15     // Alternatively, could cast p to volatile atomic...
16     depending_ptr<T> temp(*(T *volatile *)&p);
17
18     return temp;
19 }
20
21 template<typename T>
22 T* rcu_store_release(std::atomic<T*> *p, T *v)
23 {
24     p->store(v, std::memory_order_release);
25     return v;
26 }
27
28 template<typename T>
29 T* rcu_store_release(T **p, T *v)
30 {
31     // Alternatively, could cast p to volatile atomic...
32     atomic_thread_fence(std::memory_order_release);
33     *((volatile T **)p) = v;
34     return v;
35 }
36
37 // Linux-kernel compatibility macros, not for atomics
38 #define rcu_dereference(p) rcu_consume(p)
39 #define rcu_assign_pointer(p, v) rcu_store_release(&(p), v)

```

Figure 22: Dependency-Preserving Release and Consume

```

1 void *thread0(void *unused)
2 {
3     rcutest *p;
4
5     p = new rcutest();
6     assert(p);
7     p->a = 42;
8     assert(p->a != 43);
9     rcu_store_release(&gp, p);
10    return nullptr;
11 }
12
13 void *thread1(void *unused)
14 {
15     depending_ptr<rcutest> p;
16
17     p = rcu_consume(&gp);
18     if (p)
19         p->a = 43;
20     return nullptr;
21 }

```

Figure 23: Template: Simple Case

```

1 void *thread0(void *unused)
2 {
3     rcutest *p;
4
5     p = new rcutest();
6     assert(p);
7     p->a = 42;
8     rcu_store_release(&gp, p);
9     return nullptr;
10 }
11
12 void
13 thread1_help(depending_ptr<rcutest> q)
14 {
15     if (q)
16         assert(q->a == 42);
17 }
18
19 void *thread1(void *unused)
20 {
21     depending_ptr<rcutest> p;
22
23     p = rcu_consume(&gp);
24     thread1_help(p);
25     return nullptr;
26 }

```

Figure 24: Template: In via Function Parameter

```

1 void *thread0(void *unused)
2 {
3     rcutest *p;
4
5     p = new rcutest();
6     assert(p);
7     p->a = 42;
8     rcu_store_release(&gp, p);
9     return nullptr;
10 }
11
12 depending_ptr<rcutest> thread1_help()
13 {
14     return rcu_consume(&gp);
15 }
16
17 void *thread1(void *unused)
18 {
19     depending_ptr<rcutest> p;
20
21     p = thread1_help();
22     if (p)
23         p->a = 43;
24     return nullptr;
25 }

```

Figure 25: Template: Out via Function Return

<https://github.com/paulmckrcu/2016markconsume.git>.

3.5 Mark Root/Leaf Pairs

These approaches create point-to-point associations between `memory_order_consume` loads and the memory references that depend on them. Function calls can be handled by using the arguments of the function call and the function parameters as intermediate points in the association. Function returns can be handled by using the function return declaration and the function return value.

However, these point-to-point associations are required to gracefully handle bushy dependency trees, dependency trees that fan both in and out, and conditional compilation. Any scheme that relies on directly referencing a specific location in the source code will fall afoul of these requirements.

One approach is to use a unique identifier for each dependency tree, and associate each relevant point in the code with the corresponding identifiers.

Note that the root-leaf information could in theory be extracted by the compiler based on object markings (see Section 3.4).

```

1 void *thread0(void *unused)
2 {
3     rcutest *p;
4
5     p = new rcutest();
6     assert(p);
7     p->a = 42;
8     rcu_store_release(&gp, p);
9
10    p = new rcutest();
11    assert(p);
12    p->a = 43;
13    rcu_store_release(&gsgp, p);
14
15    return nullptr;
16 }
17
18 depending_ptr<rcutest>
19 thread1_help(depending_ptr<rcutest> p)
20 {
21     if (p)
22         assert(p->a == 42);
23     return rcu_consume(&gsgp);
24 }
25
26 void *thread1(void *unused)
27 {
28     depending_ptr<rcutest> p;
29
30     p = rcu_consume(&gp);
31     p = thread1_help(p);
32     if (p)
33         assert(p->a == 43);
34     return nullptr;
35 }

```

Figure 26: Template: In and Out, But Different Chains

```

1 void *thread0(void *unused)
2 {
3     rcutest *p;
4
5     p = new rcutest();
6     p->a = 42;
7     rcu_store_release(&gp, p);
8     return nullptr;
9 }
10
11 void thread1_help1(depending_ptr<rcutest> q)
12 {
13     if (q)
14         assert(q->a == 42);
15 }
16
17 void thread1_help2(depending_ptr<rcutest> q)
18 {
19     if (q)
20         assert(q->a != 43);
21 }
22
23 void *thread1(void *unused)
24 {
25     depending_ptr<rcutest> p;
26
27     p = rcu_consume(&gp);
28     thread1_help1(p);
29     thread1_help2(p);
30     return nullptr;
31 }

```

Figure 27: Template: Chain Fanning Out

```

1 void *thread0(void *unused)
2 {
3     rcutest *p;
4     rcutest1 *p1;
5
6     p = new rcutest();
7     p->a = 42;
8     rcu_store_release(&gp, p);
9
10    p1 = new rcutest1();
11    p1->a = 41;
12    p1->rt.a = 42;
13    rcu_store_release(&g1p, p1);
14
15    return nullptr;
16 }
17
18 void thread1_help(depending_ptr<rcutest> q)
19 {
20     if (q)
21         assert(q->a == 42);
22 }
23
24 void *thread1(void *unused)
25 {
26     depending_ptr<rcutest> p;
27
28     p = rcu_consume(&gp);
29     thread1_help(p);
30     return nullptr;
31 }
32
33 void *thread2(void *unused)
34 {
35     depending_ptr<rcutest1> p1;
36
37     p1 = rcu_consume(&g1p);
38     thread1_help(depending_ptr<rcutest>(&p1->rt));
39     return nullptr;
40 }

```

Figure 28: Template: Chain Fanning In

```

1 void *thread0(void *unused)
2 {
3     rcutest *p;
4     rcutest1 *p1;
5
6     p = new rcutest();
7     assert(p);
8     p->a = 42;
9     p->b = 43;
10    rcu_store_release(&gp, p);
11
12    p1 = new rcutest1();
13    assert(p1);
14    p1->a = 41;
15    p1->rt.a = 42;
16    p1->rt.b = 43;
17    rcu_store_release(&g1p, p1);
18
19    return nullptr;
20 }
21
22 void thread1a_help(depending_ptr<rcutest> q)
23 {
24     assert(q->a == 42);
25 }
26
27 void thread1b_help(depending_ptr<rcutest> q)
28 {
29     assert(q->b == 43);
30 }
31
32 void thread1_help(depending_ptr<rcutest> q)
33 {
34     if (q) {
35         thread1a_help(q);
36         thread1b_help(q);
37     }
38 }
39
40 void *thread1(void *unused)
41 {
42     depending_ptr<rcutest> p;
43
44     p = rcu_consume(&gp);
45     thread1_help(p);
46     return nullptr;
47 }
48
49 void *thread2(void *unused)
50 {
51     depending_ptr<rcutest1> p1;
52
53     p1 = rcu_consume(&g1p);
54     thread1_help(depending_ptr<rcutest>(&p1->rt));
55     return nullptr;
56 }

```

Figure 29: Template: Chain Fanning In and Out

```

1 void *thread0(void *unused)
2 {
3     rcutest *p;
4     rcutest1 *p1;
5
6     p = new rcutest();
7     assert(p);
8     p->a = 42;
9     p->b = 43;
10    rcu_store_release(&gp, p);
11
12    p1 = new rcutest1();
13    assert(p1);
14    p1->a = 41;
15    p1->rt.a = 42;
16    p1->rt.b = 43;
17    rcu_store_release(&g1p, p1);
18
19    return nullptr;
20 }
21
22 #ifdef F00
23 void thread1a_help(depending_ptr<rcutest> q)
24 {
25     assert(q->a == 42);
26 }
27 #endif
28
29 void thread1b_help(depending_ptr<rcutest> q)
30 {
31     assert(q->b == 43);
32 }
33
34 void thread1_help(depending_ptr<rcutest> q)
35 {
36     if (q) {
37         #ifdef F00
38             thread1a_help(q);
39         #endif
40         thread1b_help(q);
41     }
42 }
43
44 void *thread1(void *unused)
45 {
46     depending_ptr<rcutest> p;
47
48     p = rcu_consume(&gp);
49     thread1_help(p);
50     return nullptr;
51 }
52
53 void *thread2(void *unused)
54 {
55     depending_ptr<rcutest1> p1;
56
57     p1 = rcu_consume(&g1p);
58     thread1_help(depending_ptr<rcutest>(&p1->rt));
59     return nullptr;
60 }

```

Figure 30: Template: Conditional Compilation of Chain Endpoints

```

1 void *thread0(void *unused)
2 {
3     rcutest *p;
4
5     p = new rcutest();
6     p->a = 42;
7     assert(p->a != 43);
8     rcu_store_release(&gp, p);
9     return nullptr;
10 }
11
12 void *thread1(void *unused)
13 {
14     depending_ptr<rcutest> p;
15
16     p = rcu_consume(&gp);
17     if (p) {
18         assert(p->a == 42);
19         spin_lock(&p->lock);
20         p = std::kill_dependency(p);
21         p->a++;
22         spin_unlock(&p->lock);
23     }
24     return nullptr;
25 }

```

Figure 31: Template: Handoff to Locking

4 Evaluation

Table 1 provides a rough comparison between the various marking methods, and also includes the unmarked option for comparison purposes.

For ease of compilation, the cells corresponding to methods that explicitly mark dependency chains or that don’t require marking at all are left blank. Those that require tracing dependency chains throughout the full translation unit are marked “T” and those that limit the code in which tracing is required are marked “t”.

For ease of modification, the cells corresponding to methods that either require no marking or that mark large-scale entities are left blank. Those that require marking the definitions of objects that carry dependencies are marked “o”, and those require marking of individual accesses are marked “A”.

Cells corresponding to those methods that precisely mark dependency chains are left blank, otherwise, they are marked “N”.

For cross-function dependency chains, those methods that either support cross-function marking or that do not require such marking are left blank. Those that require manual consistency checks are

Mark:	Ease of Compilation	Ease of Modification	Precise Dependency Chains	Cross-Function Dependency Chains	Cross-Compilation-Unit Dependency Chains	C Compatibility	Formal Verification
Translation Unit	T		N		m		N
Range of Code	t		N	m	m		N
Functions	t		N	m	m		N
Objects							
Attribute		o		t	t	a	
Type Qualifier		o					N
Modifier		o		t	t		
Template		o				N	
Template+Modifier		o					
Root/Leaf		A		?	?	?	?
Nothing			N				N

Table 1: Dependency-Chain Marking Evaluation

marked “m”, those that are amenable to consistency-check tooling are marked “t”, and those that are not fleshed out sufficiently to tell are marked “?”. These same markings are used for cross-compilation-unit dependency chains.

Cells corresponding to those methods supporting C compatibility are left blank. Those that would support C compatibility if C were to provide attributes are marked “a”. Those that do not support C compatibility (at least not unless combined with some other method) are marked “N”, and those that are not fleshed out sufficiently to tell are marked “?”.

Cells corresponding to methods believed to support formal verification are left blank, those that are believed not to support formal verification are marked “N”, and those that are not fleshed out sufficiently to tell are marked “?”. Note that the object type qualifier could in theory support formal verification, but the specific proposal rules this out by requiring that the compiler treat `memory_order_consume` loads as potentially returning any value from the type.

Following the lead of C11 and C++11 atomics, the “Template+Modifier” row covers the combination of marking objects with template classes (for C++) and with an typed object modifier (for C), a combination that appears to be quite attractive. This should be further combined with a totally unmarked option for use by standalone projects such as the Linux kernel.

The best possible method would have a row of all blank cells.

5 Summary

This paper reviewed the 2016 discussions of `memory_order_consume` that took place at the Jacksonville meeting, presented several representative use cases, listed evaluation criteria, presented a number of marking proposals, and provided a comparative evaluation. The paper presents two of the marking proposals in depth, including code for the representative use cases.

We recommend a combination of typed object modifier (for C compatibility) and a template class (for C++), which is similar to the approach used by atomics. For standalone applications such as the

Linux kernel, there should additionally be an unmarked option, where the implementation assumes that everything that could legally be marked is so marked.

References

- [1] SMITH, R. Working draft, standard for programming language C++. <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2015/n4527.pdf>, May 2015.